Finite Address Component Replay and an SU(3) Singlet Audit

A conditional WRRA computation and a precise limit on address-color coupling

Wonsik Choi and Jeongin Choi
Independent Research, Seoul, Republic of Korea
20 October 2026 | v0.7
janefather@gmail.com | ORCID 0009-0001-4263-9772

Abstract

We audit the particle/component branch of Minimal Computing Cosmology 3.0 and implement a source-specific release gate that distinguishes verified replay from unsupported particle/color interpretations. Frozen-input replay reproduces 427,892 odd-composite addresses, phenotype weight 0.050000000000000065 and assembly-order total-variation distance 0.20472100116369085. The supplied antisymmetric three-color state is annihilated by all eight total generators. Exact singlet support in three fundamental factors forces address-color factorization, restricting what a color-only interface can retain. Source inspection and restricted-namespace execution establish that the original color routine reads no arithmetic state. These findings now drive an executable development decision: the delivered gate accepts conservation, declared color consistency and the conditional factorization claim, but rejects an arithmetic-origin claim and an order-independent-decoder claim even though the original 36 checks pass. The work is a bounded reproducibility and implementation note; it adds no new SU(3) representation theory, particle identification or observational validation.

1. One development question and its actual use

The question is whether a successful combined Stage-3 replay supports each proposed claim about its arithmetic component and color interfaces. Component statistics and color invariance are different outputs of the same package. The replay measures an order ambiguity relevant to the component decoder; dependency inspection tests whether the color output depends on that decoder; the kernel proof restricts what the declared color state can encode. Neither branch alone answers all three release questions.

The users of the artifact are the authors and maintainers packaging this WRRA branch for review or extending its decoder. This revision adds claim_gate.py to that actual release workflow. It prevents the package-level word PASS from being transferred to claims not exercised by the successful tests. The audited original already calls its decoder conditional; we do not attribute an arithmetic-origin claim or a deployed decoder bug to it. The implemented change is the new release decision procedure, whose acceptance and rejection cases appear in Section 4.

Assessment contract

Inputs: frozen address calibration, inherited assembly/charge rules, adopted three-color representation, and explicitly declared release-claim rules.
WRRA transformation: weighted address assembly and color checks feed a dependency and information-retention audit, followed by claim eligibility decisions.
Outputs: reproduced component statistics, color residuals, exact factorization and executable accepted/rejected claim records.
Falsification: failed replay, a violated generator/kernel identity, a counterexample to factorization under the stated support assumptions, or gate behavior inconsistent with its declared predicates rejects the corresponding implementation claim. This gate does not decide scientific merit or observational validity.

2. Frozen component replay and the supplied color branch

WRRA abbreviates Worldline-Residue-Resource-Action. The frozen upstream commit is 9d9daf21154e46e55b9fd0fd461c7c070156ab94. Active addresses are n=2,...,1,015,000, weighted as n^(-alpha), alpha=1.8996877935161325. The admission threshold h=1.4476744689338703 is retained without refitting. On odd composites, e_phi=1-product_k[1-sigmoid(d_k-h)] for k=0,...,7, where d_k=sum_j cos[gamma_j(log n+0.1k)]/2 and sigmoid(x)=1/(1+exp(-x)). Other addresses have zero phenotype effect. Adopted gamma values are 14.134725141734695, 21.022039638771556, 25.01085758014569 and 30.424876125859512.

```
p(n)=w(n)e_phi(n) / sum_m w(m)e_phi(m)
```

For each pool of the first K prime labels, initialize floor(n/sum(primes)) copies of every prime, then traverse the chosen ascending or descending order, taking one prime when the residue permits, until it is below two. Integer reconstruction n=sum_j count_j prime_j+residue is preserved. The candidate fraction is the p-weighted count of historically selected labels {5,47}, divided by the p-weighted total component count. Particle names, those labels and the 16-channel charge table are supplied. The exact rational neutral motifs are uud+electron, udd, an electron/conjugate pair and an individually neutral channel.

Branch	Candidate fraction / mean component count
K15_small	0.129471986937 / 15.9763714572
K15_large	0.127159411697 / 11.4294924701
K16_small	0.124982781217 / 15.2321441711
K16_large	0.111240636923 / 10.5785808287

For aligned component-number distributions P_j, TV=(1/2)sum_j |P_j(small)-P_j(large)|=0.20472100116369085 at K=15. Thus the tested assembly is order dependent. Fixed chosen orders remain legitimate algorithms; an order-independent decoder is a stronger claim. No observed neutron abundance is inferred from these conditional component fractions.

```
|S>=(|rgb>+|gbr>+|brg>-|rbg>-|grb>-|bgr>)/sqrt(6)
Q_a=T_a tensor I tensor I + I tensor T_a tensor I + I tensor I tensor T_a
```

Standard fundamental generators satisfy Tr(T_a T_b)=delta_ab/2. The original color_check() uses twice this normalization; annihilation and the kernel are unchanged. It constructs the three-dimensional tensor, signed permutations and eight matrices internally, and reads no address or assembly data.

Input branch	Consumer and consequence
Address measure and admission	Prime assembly; component regression and order sensitivity
Charge table and named motifs	Exact rational neutrality checks
Supplied color states and matrices	Standalone singlet test; not an arithmetic-to-color channel
Generator common kernel	Conditional information-retention constraint

dependency_audit.py recursively checks external bytecode reads and runs the extracted original function with only NumPy, itertools, math and the builtins complex, float, max, range and sum. Those eight names are the entire external-read set; no closure state is captured. This confirms the source-specific dependency finding, assuming ordinary unmodified libraries. Targeted AST extraction bypasses unrelated macro imports; this is not a full Stage-3 pipeline rerun.

3. Exact address-color restriction

Proposition. Let H_A be any finite address space and H_C=(C^3) tensor (C^3) tensor (C^3), carrying the same fundamental SU(3) action on each factor. Let P project onto the common kernel of the eight Q_a. If a density operator Omega satisfies Omega=(I_A tensor P) Omega (I_A tensor P), then P=|S><S| and Omega=sigma_A tensor |S><S| for a unique address density operator sigma_A.

Proof. The two diagonal traceless generators require equal occupation of all three colors, leaving only the six permutations of rgb. For each pair of colors, the raising operator maps two permutations differing by their exchange to the same repeated-color output. Its vanishing forces opposite coefficients. These transposition relations connect all six permutations, fixing their coefficients up to an overall amplitude and permutation parity. Direct substitution verifies the resulting |S> is annihilated by every Q_a, so the kernel has dimension one. Expand Omega=sum_nm |n><m| tensor C_nm. Rank-one support gives C_nm=P C_nm P=c_nm P. Hence Omega=sigma_A tensor P. Positivity and unit trace pass to sigma_A; partial trace gives uniqueness.

For address observable B and color observable C, expectations therefore factor as Tr(sigma_A B) times <S|C|S>. Address-controlled collective transformations U_n tensor U_n tensor U_n with U_n in SU(3) leave |S> fixed and cannot encode different address labels into this sector. A prospective decoder that deletes address labels cannot reconstruct them from this fixed color state. Exact support is stronger than zero mean charge; omitted gluon, flavor or spatial factors can change the applicable representation. Spin, confinement, binding, a particle spectrum and arithmetic selection of SU(3) remain open.

4. The implemented release decision

The three findings answer one packaging problem: which interpretation can accompany the actual Stage-3 outputs? A reproducible component replay is needed to establish the measured order dependence; dependency inspection establishes that color success does not supply arithmetic sensitivity; the rank-one proof establishes the declared color interface limit. claim_gate.py consumes audit_results.json, dependency_results.json and a JSON list of requested claim identifiers. Its predicates are explicit, source-specific release policy, not a new universal criterion of physics.

Requested claim / integration case	Delivered gate result
Component replay; declared color consistency; exact three-factor constraint	ACCEPT; all three are supported under their stated contracts
Add arithmetic_derivation_of_color	REJECT despite the original 36 checks passing: the audited routine has no arithmetic input
Add order_independent_component_decoder	REJECT using the actual nonzero assembly TV
Request an unknown claim, e.g. confinement_derived	REJECT: no declared supporting contract

The code returns process exit status 0 for an accepted requested claim set and 2 for a rejected one. The concrete development decision is therefore implemented: this release can carry its demonstrated conditional results, while an automated review/export step refuses the unsupported stronger interpretations. It does not establish that the original project made those interpretations. The factorization proof supplies a further rule for future decoder design; it does not prescribe a new physical coupling. Adding more tests of the same singlet would not establish such a coupling.

All four integration scenarios pass their expected acceptance/rejection conditions. The original 36-check suite and four dependency checks remain separately reported. The utility is a working claim boundary for this release, rather than novelty of the underlying representation theory or a claim that test counts establish physical adequacy.

5. Verification results and reproduction

Diagnostic	Recorded result
Original replay/color suite	36 / 36 pass
Source-dependency suite	4 / 4 pass
Release-gate integration scenarios	4 / 4 behave as specified
Common kernel dimension	1, numerical and exact symbolic
Maximum total-generator action	0 in the recorded original run
Generator Gram / commutator errors	1.110e-16 / 1.241e-16
Collective SU(3) rotation error, 32 trials	5.847e-16
Joint-state factorization error	1.507e-16
rgb / one-site phase negative-control charge norms	0.707106781 / 0.816496581

The exact symbolic calculation uses nonzero rescalings of the generators to remove radicals; the common kernel and signed integer singlet vector are checked without numerical tolerance. The joint-state test projects a seeded positive 6-address by 27-color density operator into the exact singlet sector and compares it with the product of its address marginal and the singlet projector. Negative controls violate the color constraint and are detected. Checks overlap and are not independent physical observations.

The exact v0.7 manuscript and companion sources are archived at https://doi.org/10.5281/zenodo.23497198. RELEASE.json identifies this PDF, scientific scripts, source manifest and reference outputs with SHA-256. SOURCE_SHA256.json is checked before scientific execution. The unchanged upstream source has commit 9d9daf21154e46e55b9fd0fd461c7c070156ab94; its Stage-3 code and calculations/wrra_m_0_9 are reproduced in source/.

```
python3 -m venv .venv
.venv/bin/python -m pip install numpy==2.3.5 scipy==1.17.0 sympy==1.14.0
.venv/bin/python audit.py
.venv/bin/python dependency_audit.py
.venv/bin/python claim_gate.py
```

Recorded Python: 3.12.14. Reruns overwrite their result JSON files; preserve reference bytes for release-hash checks. Different runtime metadata or numerical libraries can change output hashes. Absolute fraction/share tolerances are 2 x 10^-12, historical mean-count tolerance 2 x 10^-10, matrix tolerance 10^-12 and numerical kernel threshold 10^-10; symbolic tests use exact zero. Gate acceptance requires the declared audit statuses and source-specific evidence contracts. Known inputs and reproduced values remain legitimate model-construction and consistency results. This note does not require a new observational prediction to state its bounded contribution.

Conclusion

The frozen particle/component replay is reproducible and order dependent. Its supplied color test preserves the declared singlet but consumes no arithmetic state, and its exact three-factor singlet sector cannot retain address distinctions. The delivered release gate makes those findings usable in the actual package: it accepts supported conditional statements and rejects stronger unsupported interpretations. This is a compact reproducibility and technical implementation report; general physical completion remains outside its claim.

Sources and established mathematical background

[1] W. Choi and J. Choi, frozen WRRA Stage-3 particle/component replay and Stage-2 handoff, repository github.com/Wonsik-Choi-janefather/wrra-m-0.1, commit 9d9daf21154e46e55b9fd0fd461c7c070156ab94. Exploratory source DOI: 10.5281/zenodo.23128482. These are computational provenance, not external experimental replication.

[2] D. Tong, The Standard Model, chapter 3, The Strong Force, University of Cambridge lecture notes, equations (3.4)-(3.6) and (3.95). https://davidtong.org/pdfs/teaching/standard-model/standardmodel3.pdf. Standard SU(3) and epsilon-singlet background.